

PhishGuard

Email and SMS Phishing Detection Models

Complete Training Guide: models, datasets (Kaggle and other public sources), hand-made features, every threshold, step-by-step training and expected results

Item	Value
Repository	https://github.com/Pranavvvv-09/AI-Based-Phishing-Detection-Tool
Branch	claude/blissful-dijkstra-fheyhs
Code verified at	commit 2a14dfa (Day 6 + SMS false-positive fix)
Models	Email and SMS: TF-IDF (word 1-2 grams) + Logistic Regression (scikit-learn 1.9.1). SMS wording is scored by averaging the SMS and email models. An explainable scorer combines the text score with 18 header rules and 27 link/attachment rules.
Training data	Email: 6,125 phishing + 30,304 legitimate. SMS: 347 smishing + 4,021 legitimate. Each model also gets the 56-message train half of a hand-written modern set.
Guide date	7 October 2026

How this guide was verified. Every Linux command in sections 4, 5 and 17 was executed on a **fresh clone** of the branch above at commit 2a14dfa with Python 3.13.16: install, download and hash checks (one corrupted transfer was retried automatically), training of both models, the full-system evaluation, the SMS experiment, the ablation study, the tests and the post-training commands. The Linux manual-download commands in section 9 were run in an empty folder, and every file matched its pin. The freshly trained models and evaluations reproduced every number in sections 15 and 16 **exactly**, and the test suite passed (269 tests; also on Python 3.11.17). Windows and macOS commands are given but were **not** executed by the author.

Contents

1. What you are training
2. Requirements
3. Safety rules (read first)
4. Quick start (one command)
5. Step-by-step training
6. Datasets used: catalogue and links
7. More public datasets (Kaggle and others, not used)
8. Integrity pins (hashes and commit)
9. Manual download (if the script cannot run)
10. How the data is prepared
11. Features (trained and hand-made)
12. Every threshold and fixed number
13. Model and training settings
14. How the models are evaluated
15. Expected results
16. The SMS false-positive experiment
17. Using and inspecting your model
18. Adding your own examples
19. Troubleshooting
20. Metric glossary
21. Known limitations
22. Licences and attribution

1. What you are training

PhishGuard judges a message with independent layers. **Only the two text models are trained** (one for email, one for SMS). The header and link layers are fixed, hand-made rules. The **scorer** combines everything into one explainable verdict (section 11.5).

Layer	Module	Type	What it judges
Email text model	text_model.py	Machine learning (trained)	Phishing intent in subject + body: urgency, requests to verify, pay, transfer, reply with codes
SMS text model	text_model.py	Machine learning (trained)	Smishing intent in an SMS (digits ignored). Its score is averaged with the email model's score (section 16)
Header checks	header_checks.py	Rules (18 checks)	SPF/DKIM/DMARC results, spoofed display names, Reply-To tricks, lookalike sender domains
URL & attachment checks	url_checks.py	Rules (27 checks)	Lookalike / punycode / IP links, link-text mismatch, embedded forms, risky attachments
Scorer	scorer.py	Fixed formula	Adds all evidence in log-odds, applies the thresholds, explains why

Why the text models ignore links and sender addresses: they are judged by the rule layers, and leaving them in the text taught an early model the shortcut "has a link = phishing" (section 10).

Pipeline

```
EMAIL raw .eml -> safe parser (size limits, no HTML rendering)      parser.py
      -> subject + visible body text                                email_text()
      -> normalise: drop URLs/emails, numbers -> numtoken, lower-case  normalize_text()
      -> TF-IDF word 1-2 grams -> Logistic Regression -> probability   email_model
      + header rules + link/attachment rules + trust checks          header_checks / url_checks
      -> scorer: score, label, action and reasons                    scorer.py

SMS    text -> normalize_sms (digits dropped) -> SMS model          -> p_sms
      -> normalize_text -> email model -> p_email
      text logit = (logit(p_sms) + logit(p_email)) / 2              BlendedText
      + link rules on URLs and bare domains (e.g. sbi-kyc-update.in)  check_text_links()
      -> scorer: score, label, action and reasons
```

2. Requirements

Requirement	Minimum	Notes
Operating system	Linux, macOS or Windows 10/11	Tested on Linux. On Windows, WSL2 or a VM is recommended (section 3).
Python	3.11 or newer	Tested with 3.11.17 and 3.13.16.
git	any recent version	Required: the honeypot dataset is fetched with git.
Free disk space	4 GB	Measured: data/raw 2.6 GB, data/processed 65 MB, .venv 428 MB.
RAM	2 GB free	Measured peak during download + training: 1.14 GB.
Internet access	github.com, raw.githubusercontent.com, pypi.org	HTTPS only.
Time	about 6-7 minutes (first run)	Measured on the fresh clone: install 23 s; 5 min 53 s for download + parsing + training both models; full-system evaluation 4 min 54 s. Retraining later: email model 2 min 31 s, SMS model 2 s.

3. Safety rules (read first)

The training data contains **real phishing emails**, and the honeypot set contains **real malicious attachments**. Handled as described here they are harmless, because PhishGuard only reads them as text.

- **Never open** any file in data/raw/ with a mail client, browser, Word or a PDF reader. Never click links in them.

- PhishGuard parses them **statically** with its hardened parser (12,269 real honeypot emails parsed with 0 crashes).
- Work in a virtual environment; preferably inside a VM or WSL2 so the samples stay off your daily-use system.
- **Antivirus:** it may quarantine some honeypot files. That is expected and safe; do **not** add exclusions. Your sample count, and therefore your metrics, may then differ slightly from section 15.
- data/raw/ and data/processed/ are gitignored. Never commit or share them.
- The honeypot set is licensed **CC BY-NC 4.0**: non-commercial use only.
- Never print or log full email bodies (the code never does).
- Use PhishGuard only on mailboxes and messages you own or are allowed to test.

4. Quick start (one command)

Linux / macOS

```
git clone -b claude/blissful-dijkstra-fheyhs \
    https://github.com/Pranavvvv-09/AI-Based-Phishing-Detection-Tool.git
cd AI-Based-Phishing-Detection-Tool
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
python scripts/bootstrap.py
```

Windows (PowerShell)

```
git clone -b claude/blissful-dijkstra-fheyhs `
    https://github.com/Pranavvvv-09/AI-Based-Phishing-Detection-Tool.git
cd AI-Based-Phishing-Detection-Tool
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -e ".[dev]"
python scripts\bootstrap.py
```

bootstrap.py downloads and verifies all datasets, then trains the email and SMS models, but only those that are missing or invalid. Re-running it is safe; it then prints [ok] email_model and sms_model present and match models/manifest.json - nothing to do. Use python scripts/bootstrap.py --force to retrain anyway. If PowerShell blocks Activate.ps1, run Set-ExecutionPolicy -Scope CurrentUser RemoteSigned once.

5. Step-by-step training

Use this instead of the quick start if you want to see and check each stage.

Step 1 - Get the code

```
git clone -b claude/blissful-dijkstra-fheyhs \
    https://github.com/Pranavvvv-09/AI-Based-Phishing-Detection-Tool.git
cd AI-Based-Phishing-Detection-Tool
```

In Windows PowerShell, end the first line with a backtick (`) instead of a backslash, as in section 4.

Step 2 - Create a virtual environment and install

```
python3 -m venv .venv # Windows: python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -e ".[dev]"
```

This installs PhishGuard plus pandas, scikit-learn, tldextract, joblib, pytest, ruff, bandit and pip-audit.

Step 3 - Download and verify the datasets

```
python scripts/download_data.py
```

Expected output (first run). Every line must be [done] or [ok]:

```
[done] Nazario.csv: 7.8 MB
[done] Nigerian_Fraud.csv: 9.2 MB
[done] Enron.csv: 45.6 MB
[done] SpamAssasin.csv: 14.9 MB
```

```
[done] CEAS_08.csv: 67.9 MB
[done] Ling.csv: 9.3 MB
[done] sms.tsv: 0.5 MB
[done] sms_mendeley_5971.csv: 0.6 MB
[done] phishing_pot at pinned commit 89e2bc05d159
```

A [retry] line means a transfer was corrupted or cut short and is being downloaded again (this happened once during the verification run). A file is accepted **only** if its SHA-256 matches the pin. A [fail] line after 3 attempts means: do not train; see section 19.

Step 4 - Train the email model

```
python -m phishguard.text_model train-email
```

The first run parses about 12,000 honeypot emails (about 2 minutes) and caches the extracted text in data/processed/. Then it trains the final model plus four leave-one-corpus-out probe models. The first output line must be exactly (shown wrapped here; it is one line):

```
corpus after de-duplication: {'CEAS_08.csv:0': 16968, 'Enron.csv:0': 14444,
'Ling.csv:0': 2396, 'Nazario.csv:1': 1526, 'Nigerian_Fraud.csv:1': 3200,
'SpamAssasin.csv:0': 4073, 'phishing_pot:1': 3349}
```

The last line must be saved models/email_model.joblib (sha256 recorded in models/manifest.json).

Step 5 - Train the SMS model

```
python -m phishguard.text_model train-sms
```

Takes about 2 seconds. The first output line must be exactly:

```
sms corpus after de-duplication: {'sms.tsv:0': 194, 'sms_mendeley_5971.csv:0': 4832,
'sms_mendeley_5971.csv:1': 434}
```

The scorer always uses **both** models for SMS, so train both before scoring SMS.

Step 6 - Verify

```
pytest # expected: 269 passed
python scripts/bootstrap.py # expected: [ok] ... present and match models/manifest.json - nothing to do
```

Quality tests check your trained models against fixed bars (section 12, table "Quality bars"). If they pass, your models meet the documented quality.

Step 7 - Evaluate the complete system

```
python scripts/evaluate_scorer.py
```

Runs held-out messages through the full scorer exactly as the app will (real honeypot .eml files with their headers, legitimate test emails, the modern test, the SMS test split and the frozen SMS sets) and writes models/scorer_metrics.json. Takes about 4 min 54 s. Results: section 15.7.

Step 8 - Optional: re-run the SMS experiment (about 4 seconds)

```
python scripts/sms_experiment.py
```

Retrains the four SMS variants, applies the pre-registered rule and writes models/sms_experiment.json. Expected output:

```
val F1 val FP test FPR test rec indep FP mod FPR pub rec
V0 0.667 6 0.375 0.917 2 0.214 0.966
V1 0.741 5 0.250 0.958 3 0.071 0.954
V2 0.833 2 0.208 1.000 0 0.107 0.849
V3 0.750 3 0.208 1.000 2 0.214 0.908
eligible: ['V0', 'V1', 'V3'] | choice: V3 | wrote models/sms_experiment.json
```

Columns are explained in section 16.

Step 9 - Optional: ablation study (about 1.5 minutes)

```
python scripts/ablation.py
```

Trains three email models on different data and scores them on identical tests (section 15.5).

Files produced

Path	Content	In git?
data/raw/	Downloaded datasets (about 2.6 GB)	No (gitignored)
data/processed/phishing_pot_v2_*.csv	Cached normalised honeypot text	No (gitignored)
models/email_model.joblib	The trained email model (a pickle)	No (gitignored)
models/sms_model.joblib	The trained SMS model (a pickle)	No (gitignored)
models/manifest.json	Per model: SHA-256, normaliser, training time, sklearn version, metrics	Yes
models/scorer_metrics.json	Full-system evaluation results	Yes
models/sms_experiment.json	SMS experiment: all variants, eligibility, choice	Yes

Security note: .joblib files are pickles, and loading a pickle can execute code. `load_model()` refuses to load a model unless its SHA-256 matches `models/manifest.json`. Never load a model file someone sends you. **Your SHA-256 will differ from the author's** even though the learned model is bit-identical: scikit-learn stores a memory address (`_stop_words_id`) inside the saved vectorizer. Your metrics must match; your hashes will not.

6. Datasets used: catalogue and links

All files are downloaded automatically by `scripts/download_data.py`. Labels in the raw CSVs: 1 = phishing/spam, 0 = legitimate. PhishGuard uses **only** the rows listed under "Rows used" (spam rows are excluded because spam is not phishing).

Dataset	Raw rows (label 0 / 1)	Rows used (after de-dup)	Columns (raw features)	Our label
Nazario.csv	1,565 (0 / 1,565)	1,526: all	sender, receiver, date, subject, body, urls, label	phishing
Nigerian_Fraud.csv	3,332 (0 / 3,332)	3,200: all	sender, receiver, date, subject, body, urls, label	phishing
phishing_pot (honeypot)	12,271 .eml files	3,349: English only	full raw email (.eml): headers, body, attachments	phishing
Enron.csv	29,767 (15,791 / 13,976)	14,444: label 0	subject, body, label	legitimate
SpamAssasin.csv	5,809 (4,091 / 1,718)	4,073: label 0	sender, receiver, date, subject, body, label, urls	legitimate
CEAS_08.csv	39,154 (17,312 / 21,842)	16,968: label 0	sender, receiver, date, subject, body, label, urls	legitimate
Ling.csv	2,859 (2,401 / 458)	2,396: label 0	subject, body, label	legitimate
sms_mendeley_5971.csv	5,971 (ham 4,844 / smishing 638 / spam 489)	4,832 ham + 434 smishing	LABEL, TEXT, URL, EMAIL, PHONE	SMS legit / smishing
sms.tsv	5,574 (ham 4,827 / spam 747)	194 ham (UCI-only)	label, text (tab-separated)	SMS legit

Only the **subject** and **body** columns (or the parsed subject + visible body of an .eml) are used by the email model; only the SMS text by the SMS model.

Hand-made files in the repository (data/curated/)

File	Rows (legit / phishing)	Purpose	Used for
modern_lures.csv	112 (56 / 56)	Modern phishing and alarming-but-genuine messages, 27 categories	Train half: both models. Test half: modern test only
sms_transactional_eval.csv	72 (36 / 36)	Genuine bank/UPI/OTP/delivery SMS and smishing in the same formats	Evaluation only (frozen): 24 validation, 48 test
sms_independent_bank.csv	12 (12 / 0)	Real-format bank SMS written by other developers (MIT licence)	Evaluation only (frozen)
sms_templates.csv	663 (321 / 342)	Synthetic transactional SMS from 88 templates in 13 formats	Experiment variants V1/V2 only; not used by the shipped model

Download links (exactly what the script fetches)

<https://raw.githubusercontent.com/rokibulroni/Phishing-Email-Dataset/main/Nazario.csv>
https://raw.githubusercontent.com/rokibulroni/Phishing-Email-Dataset/main/Nigerian_Fraud.csv
<https://raw.githubusercontent.com/rokibulroni/Phishing-Email-Dataset/main/Enron.csv>
<https://raw.githubusercontent.com/rokibulroni/Phishing-Email-Dataset/main/SpamAssassin.csv>
https://raw.githubusercontent.com/rokibulroni/Phishing-Email-Dataset/main/CEAS_08.csv
<https://raw.githubusercontent.com/rokibulroni/Phishing-Email-Dataset/main/Ling.csv>
<https://raw.githubusercontent.com/justmarkham/pycon-2016-tutorial/master/data/sms.tsv>
https://raw.githubusercontent.com/nmbenton/INF0-4360-Project/main/Dataset_5971.csv
https://github.com/rf-peixoto/phishing_pot.git (folder email/ only)

Original sources

Dataset	Original source / citation	Link
Email CSVs (all six)	Per-source files of the Kaggle "Phishing Email Dataset" uploaded by Naser Abdullah Alam. Its description asks users to cite two articles, one of them Al-Subaiey et al. (2024), "Novel Interpretable and Robust Web-based AI Platform for Phishing Email Detection". Mirrored on GitHub.	kaggle.com/datasets/naserabdullahalam/phishing-email-dataset github.com/rokibulroni/Phishing-Email-Dataset
Nazario	Jose Nazario phishing corpus	monkey.org/~jose/phishing/
Nigerian_Fraud	CLAIR collection of fraud email (Radev)	via the Kaggle compilation
Enron	Enron-Spam corpus (Metsis, Androutsopoulos and Paliouras, CEAS 2006)	www2.aueb.gr/users/ion/data/enron-spam/
SpamAssassin	Apache SpamAssassin public corpus	spamassassin.apache.org/old/publiccorpus/
CEAS_08	CEAS 2008 Live Spam Challenge corpus	via the Kaggle compilation
Ling	Ling-Spam corpus (Androutsopoulos et al., 2000)	via the Kaggle compilation
phishing_pot	Real phishing caught by honeypots, 2022-2026 (R. Peixoto)	github.com/rf-peixoto/phishing_pot
sms.tsv	UCI SMS Spam Collection (Almeida and Hidalgo, 2011)	archive.ics.uci.edu/dataset/228/sms+spam+collection
sms_mendeley_5971.csv	SMS Phishing Dataset (Mishra and Soni, 2022), Mendeley Data, mirrored on GitHub	data.mendeley.com/datasets/f45bkkt8pr/1 github.com/nmbenton/INF0-4360-Project
sms_independent_bank.csv	Example SMS from two MIT-licensed open-source parsers (see THIRD_PARTY_NOTICES.md)	github.com/Pavel401/transaction_sms_parser github.com/saurabhgupta050890/transaction-sms-parser

The GitHub links were opened and downloaded by the author. The Kaggle, UCI, Mendeley, monkey.org, aueb.gr and apache.org links come from search results and were **not opened**: those hosts were blocked in the author's environment, which is also why public GitHub mirrors are used. The mirrors are third-party copies; the pins in section 8 guarantee that you get the exact same bytes as the author, not that a mirror is authentic.

7. More public datasets (Kaggle and others, not used)

These public datasets are **not** used by PhishGuard; they are candidates if you want more training data. None was opened or downloaded by the author (the hosts were blocked); descriptions come from search results and papers, so check each page yourself.

Dataset	What it contains (as described publicly)	Link	Before using it
SMS Spam Collection (Kaggle copy)	The same 5,574 UCI SMS labelled ham/spam that PhishGuard already uses (as sms.tsv)	kaggle.com/datasets/uciml/sms-spam-collection-dataset	Adds nothing new. Spam is not smishing: relabel or exclude spam
Phishing Email Detection (Kaggle, subhajournal)	Email text with a type label for phishing detection; size and columns not verified	kaggle.com/datasets/subhajournal/phishingemails	Licence and origin unverified; likely overlaps the classic corpora
Hugging Face phishing-dataset (ealvaradob)	Mixed phishing and benign texts: URLs, website HTML, SMS and emails (as described by projects that use it); size not verified	huggingface.co/datasets/ealvaradob/phishing-dataset	Licence not verified. Keep only SMS/email rows for the text models

Dataset	What it contains (as described publicly)	Link	Before using it
SmishTank (Timko and Rahman, CODASPY 2024)	1,062 smishing messages reported by the public to smishtank.com, with sender, brands and URL details	arxiv.org/abs/2402.18430	Smishing only (no legitimate SMS). Download access and licence unverified
Enron-Spam (original)	The original Enron-Spam folders behind Enron.csv	www2.aueb.gr/users/ion/data/enron-spam/	Already used via the compilation: duplicates
SpamAssassin public corpus (original)	easy_ham, hard_ham and spam folders behind SpamAssasin.csv	spamassassin.apache.org/old/publiccorpus/	Already used via the compilation: duplicates

Checklist before adding any dataset. (1) Read its licence; respect non-commercial terms. (2) Map its labels to phishing vs legitimate: spam and marketing are **not** phishing. (3) De-duplicate it against the existing data: many compilations re-package Enron, SpamAssassin and Nazario, and a copy of a test message in the training data inflates every score (leakage). (4) Check the language and era of each class (a class that is all-German or all-2020 teaches a shortcut). (5) Retrain, inspect the top-weighted words (section 17) and re-run the full evaluation.

8. Integrity pins (hashes and commit)

These values are hard-coded in `scripts/download_data.py`. The script refuses any file that does not match.

```
Nazario.csv          b8fbc4158fbdaff1ed98584c43d72e283c6352d7ade4d457d34c1d79488d184
Nigerian_Fraud.csv   f5d6930763cae6feeae6484131c79655a7440abbb97d6f51eab757aea4e6ab4b
Enron.csv            99933d3233510cf5dc2ee7768ddc609425cb8002bf5d7691d9a7157ffa5fd318
SpamAssasin.csv      3bfe8f8abff89f69a98456be50413c2fcb20a476141116dd84ee1507b980c00e
CEAS_08.csv          22375e7d5f5a8229dbe987914ee9b3705656c590038662a7df6054629b376074
Ling.csv             c133792260f18b251e9377b9cb31bef226322af6dc0d79841f61c67489929eca
sms.tsv              7d039a24a6083ed9ef0f806ebad56bbb976e3aeb8de05669173bfdc4996c239d
sms_mendeley_5971.csv 649844f1c62a6b27e145eaf17a65f7010c56c390e11a794ea8329993a05ba71e
phishing_pot         git commit 89e2bc05d159555389782f2f2f8d916588cd49cd
```

Additional download protections: HTTPS only (the downloader has no `http:`, `file:` or `ftp:` handler, so even a redirect cannot downgrade), 120 MB size cap per file, truncation check against Content-Length, up to 3 attempts, atomic writes (a partial file never looks valid), and git transport restricted to HTTPS.

9. Manual download (if the script cannot run)

Only needed if `download_data.py` cannot run on your machine. The files must end up exactly here:

```
data/raw/Nazario.csv  data/raw/Nigerian_Fraud.csv  data/raw/Enron.csv
data/raw/SpamAssasin.csv  data/raw/CEAS_08.csv  data/raw/Ling.csv
data/raw/sms.tsv  data/raw/sms_mendeley_5971.csv
data/raw/phishing_pot/email/sample-*.eml
```

Linux / macOS

```
mkdir -p data/raw && cd data/raw
M=https://raw.githubusercontent.com/rokibulroni/Phishing-Email-Dataset/main
for f in Nazario.csv Nigerian_Fraud.csv Enron.csv SpamAssasin.csv CEAS_08.csv Ling.csv; do
  curl -fsSL -o "$f" "$M/$f"
done
curl -fsSL -o sms.tsv \
  https://raw.githubusercontent.com/justmarkham/pycon-2016-tutorial/master/data/sms.tsv
curl -fsSL -o sms_mendeley_5971.csv \
  https://raw.githubusercontent.com/nmbenton/INF0-4360-Project/main/Dataset_5971.csv
sha256sum *.csv sms.tsv      # macOS: shasum -a 256 *.csv sms.tsv
```

Compare every hash with section 8. Example of an automatic check for one file:

```
echo "7d039a24a6083ed9ef0f806ebad56bbb976e3aeb8de05669173bfdc4996c239d  sms.tsv" | sha256sum -c -
```

Honeypot (run inside `data/raw`):

```
git init phishing_pot && cd phishing_pot
git remote add origin https://github.com/rf-peixoto/phishing_pot.git
git sparse-checkout set --no-cone email/
git fetch --depth 1 --filter=blob:none origin 89e2bc05d159555389782f2f2f8d916588cd49cd
git checkout 89e2bc05d159555389782f2f2f8d916588cd49cd
git rev-parse HEAD      # must print the commit above
```


Windows (PowerShell)

```
New-Item -ItemType Directory -Force data\raw | Out-Null; Set-Location data\raw
$M = "https://raw.githubusercontent.com/rokibulroni/Phishing-Email-Dataset/main"
foreach ($f in "Nazario.csv","Nigerian_Fraud.csv","Enron.csv","SpamAssasin.csv",
               "CEAS_08.csv","Ling.csv") { Invoke-WebRequest "$M/$f" -OutFile $f }
$S = "https://raw.githubusercontent.com/justmarkham/pycon-2016-tutorial/master/data/sms.tsv"
Invoke-WebRequest $S -OutFile sms.tsv
$S = "https://raw.githubusercontent.com/nmbenton/INF0-4360-Project/main/Dataset_5971.csv"
Invoke-WebRequest $S -OutFile sms_mendeley_5971.csv
Get-FileHash *.csv, sms.tsv -Algorithm SHA256
```

The honeypot git commands are the same in PowerShell. Afterwards run python scripts/download_data.py once: it should report every file as [ok] ... already present and verified and the honeypot as [ok] phishing_pot already present at pinned commit 89e2bc05d159.

10. How the data is prepared

Email data

All of this happens automatically inside `load_email_corpus()` and `load_honeypot()`.

- **Label filtering:** phishing = all Nazario and Nigerian_Fraud rows + English honeypot emails; legitimate = label-0 rows of Enron, SpamAssassin, CEAS_08 and Ling. Spam rows are dropped.
- **Text used:** subject + newline + body (for .eml files: parsed subject + visible text from plain-text and HTML parts; scripts and styles dropped; attachments never read as text).
- **Normalisation** (identical at training and prediction time): text cut to 40,000 characters; URLs (`http://`, `https://`, `www.`) and email addresses **removed**; every number replaced by numtoken; whitespace collapsed; lower-cased; capped at 20,000 characters.
- **Honeypot placeholder:** the corpus replaces victims' addresses with `phishing@pot`; it is removed before normalisation.
- **English filter (honeypot only):** a message is kept if it contains at least 5 of these 32 English function words and they make up at least 8% of its tokens: `all`, `an`, `and`, `are`, `as`, `at`, `be`, `by`, `can`, `for`, `from`, `has`, `have`, `if`, `in`, `is`, `it`, `not`, `of`, `on`, `or`, `our`, `please`, `that`, `the`, `this`, `to`, `we`, `will`, `with`, `you`, `your`. Only about 43% of the honeypot is English (estimated from a 1-in-5 sample); using all of it would teach "non-English = phishing".
- **Short texts:** rows with fewer than 20 characters after normalisation are dropped.
- **De-duplication:** rows whose first 2,000 normalised characters are identical are removed; the first copy wins, so a text can never appear twice, with the same or with conflicting labels.
- **Curated examples:** `data/curated/modern_lures.csv` (112 hand-written modern messages, 27 categories). The train half is added with sample weight 20; the test half is only used for evaluation.

SMS data

- **Labels:** legitimate = ham; positive = **smishing** only. Marketing spam is excluded, as for email. Where Mendeley and UCI share a message (4,933 do), the Mendeley label wins: it split UCI's "spam" into smishing and marketing.
- **Parsing:** `sms.tsv` is read with `quoting=csv.QUOTE_NONE`. The default parser treats stray quote characters as quoting and merges messages (5,572 rows instead of the real 5,574).
- **Digits are dropped** (`normalize_sms`): 97% of smishing but only 16% of the 2011-era legitimate chats contain digits. Modern legitimate SMS (OTPs, bank alerts) are full of numbers, so numbers must not mean "scam".
- **Placeholders removed:** UCI's anonymisation markers (`<#>`, `<TIME>`, ...) occur in 5% of legitimate and 0% of spam messages, so they would identify the class.
- **De-duplication** as for email; without digits many smishing messages turn out to be one campaign template, so 638 smishing rows become 434 unique messages.
- **Curated examples:** the train half is added with weight 3 (about 4% influence), fixed before training.
- **Rejected dataset:** an "Indian SMS spam" set (junioralive, MIT) labels all its credit, OTP and delivery messages as spam and has only personal chat as ham; it would have made false positives worse.

- The normaliser used is recorded in `models/manifest.json`, so the scorer can never pair a model with the wrong text processing.

11. Features (trained and hand-made)

PhishGuard has two kinds of features: the **learned** text features (11.1) and **hand-made** rules (11.2-11.4), each with a fixed weight chosen in advance. Rules are never trained, so they cannot learn a dataset quirk, and every rule that fires is shown to the user.

11.1 Text model features (what the trained models see)

Setting	Value	Meaning
Feature type	TF-IDF word n-grams	Each word and word pair gets a weight: frequent in this message, rare overall = high
ngram_range	(1, 2)	Single words and two-word phrases, e.g. your account, click here
token_pattern	(?u)\b[a-z][a-z]+\b	Words of 2+ letters (numbers are already numtoken or dropped)
min_df / max_df	3 / 0.9	Ignore terms in fewer than 3 messages or in more than 90% of messages
max_features	50,000	Vocabulary size limit
sublinear_tf	True	Uses $1 + \log(\text{term count})$, so repetition is not over-rewarded
strip_accents	unicode	é becomes e, etc.
stop_words	51 corpus-artefact words (SMS: +2)	Removed because they identify a dataset, not phishing (list below)

Stop words (corpus artefacts): enron, vince, kaminski, louise, daren, houston, ect, hou, jeff, monkey, jose, nazario, utf, iso, charset, http, https, www, com, net, org, html, htm, nbsp, subject, cc, fw, fwd, url, pot, phishing, unsubscribe, preferences, copyright, reserved, app, apps, ai, de, para, que, da, em, la, el, und, der, die, enviado, assunto, remetente

SMS model: identical settings, plus digits removed before tokenising and two extra stop words: uk, covid (the 2011 spam is British; only 2020-22 scams mention covid, so genuine vaccination SMS would otherwise be flagged).

Why these words: enron, vince, kaminski, louise, daren, houston, ect, hou, jeff are Enron names and places; monkey, jose, nazario come from the Nazario mailbox; utf, iso, charset, nbsp, html, htm are encoding residue; http, https, www, com, net, org, url are leftovers of broken-up URLs; pot, phishing come from the honeypot placeholder (and "phishing" never appears in the legitimate corpora); unsubscribe, preferences, copyright, reserved, app, apps, ai only appear in modern mail, which in this data is always phishing (era shortcut); the rest are Portuguese, Spanish and German residue (language shortcut). Each was found by inspecting the model's top-weighted words.

11.2 Header rules (hand-made; email only)

Rule (code)	Weight	Fires when
dmARC_fail	0.35	DMARC result is fail (sender domain did not authorise this message)
spf_fail	0.20	SPF result is fail or permerror
spf_softfail	0.10	SPF result is softfail
dkim_fail	0.15	DKIM signature invalid (fail or permerror)
forged_auth_header	0.30	A lower Authentication-Results header from the trusted server claims a pass the top header denies
auth_unverified	0.05	No Authentication-Results header, or the top one is not from TRUSTED_AUTHSERV_ID
reply_to_mismatch	0.25	Reply-To domain differs from the From domain
reply_to_freemail	0.10	Replies redirected to a free-mail domain while the sender is not free mail
return_path_mismatch	0.05	Bounce (Return-Path) domain differs from From (skipped for official brand senders)
display_name_brand_spoof	0.35	Display name names one of 51 brands, but the address is not on that brand's official domains (or is free mail)
display_name_other_email	0.30	Display name contains a different email address than the real sender
freemail_impersonation	0.25	Free-mail sender whose display name claims a role (one of 26 role phrases, 11.4)
lookalike_sender_domain	0.40	Sender domain imitates a brand: homoglyphs, digit swaps, typos, brand + extra words
punycode_sender_domain	0.20	Sender domain contains an IDN (xn--) label
bulk_recipients	0.05	10 or more visible To + Cc recipients
undisclosed_recipients	0.05	No visible recipients (hidden BCC mass mailing)
missing_from	0.20	No valid From address
missing_message_id	0.05	Message-ID header missing

Header risk = sum of fired weights, capped at 1.0. SPF/DKIM/DMARC are read from the receiving server's Authentication-Results header (no DNS lookups). Only the top-most header from TRUSTED_AUTHSERV_ID (default mx.google.com) may credit a pass; failures always count.

11.3 URL and attachment rules (hand-made)

Rule (code)	Weight	Fires when
url_dangerous_scheme	0.40	javascript:, data:, vbscript: or file: link
url_ip_host	0.30	Link host is an IP address
url_obfuscated_ip	0.40	Decimal, hex or octal IP host (e.g. 3232235777, 0x7f.1)
url_at_symbol	0.35	user@host trick: https://paypal.com@evil.test goes to evil.test
url_obfuscated_host	0.25	Backslash, %-encoding or invisible/bidi characters in the URL
url_punycode	0.30	IDN host (xn-- or raw Unicode such as Cyrillic look-alikes)
url_lookalike_domain	0.40	Domain imitates a brand (homoglyph, typo, brand + words)
url_brand_outside_domain	0.30	Brand named in the subdomain or path of someone else's site
url_shortener	0.10	Link shortener (19 known, e.g. bit.ly); weak signal
url_free_hosting	0.15	Free site builder or tunnel (22 known, e.g. netlify.app, ngrok)
url_cloud_user_content	0.10	User-uploaded cloud content (15 hosts: S3, GCS, Firebase, Azure, Google Forms, IPFS)
url_risky_tld	0.10	High-abuse TLD (22 listed, e.g. .zip .top .xyz)
url_http_only	0.05	Explicit unencrypted http:// (not for scheme-less bare domains)
url_many_subdomains	0.10	4 or more labels before the registered domain
url_nonstandard_port	0.10	Port other than 80/443 (or invalid)
url_credential_words	0.10	One of 21 credential words (login, verify, kyc, ...) in the path or query, unless the site is the brand's own
url_long	0.05	URL longer than 100 characters
url_malformed	0.10	URL cannot be parsed
link_text_mismatch	0.45	Visible link text shows one domain, the link goes to another (strongest signal)
link_text_brand_mismatch	0.25	Link text names a brand, the link goes elsewhere
form_in_email	0.35	HTML form inside the email (credential harvesting)
att_executable	0.50	Executable, script or disk-image attachment (24 extensions)
att_html	0.35	HTML or SVG attachment (local phishing page)
att_macro	0.35	Macro-enabled Office file (.docm, .xlsm, ...)
att_double_extension	0.30	Double extension such as invoice.pdf.exe
att_rtlo_filename	0.40	Right-to-left-override character disguising the extension
att_archive	0.15	Archive attachment (.zip, .rar, .7z, ...)

Link/attachment risk = sum of fired weights (each code counted once per message), capped at 1.0. Links are never fetched or resolved (no SSRF, no tipping off the attacker). Email click-tracking links of known mailing providers do not count as a link-text mismatch when the shown domain is the sender's own.

11.4 The hand-maintained lists behind the rules

List	Entries
Brands (51), each with its official domains	adobe, airtel, amazon, apple, axis, bankofamerica, bankofbaroda, binance, blueart, canarabank, chase, coinbase, delhivery, dhl, docusign, dropbox, epfo, facebook, fedex, flipkart, google, hdfc, icici, incometax, indiapost, instagram, irtctc, jio, kotak, lic, linkedin, metamask, microsoft, netflix, npc, paypal, paytm, phonepe, pnb, rbi, sbi, spotify, twitter, uidai, ups, usaa, usps, wells Fargo, whatsapp, yesbank, zoom
Free-mail domains (16): never trusted	aol.com, gmail.com, gmx.com, googlemail.com, hotmail.com, icloud.com, live.com, mail.com, onmicrosoft.com, outlook.com, proton.me, protonmail.com, rediffmail.com, yahoo.co.in, yahoo.com, zoho.com
Role phrases (26) for freemail_impersonation	accounts, admin, administrator, bank, ceo, cfo, court, customer care, dean, director, finance, help desk, helpdesk, hr, human resources, it department, it support, managing director, md, payroll, police, principal, registrar, security, support, tax
Link shorteners (19)	bit.ly, bitly.com, bl.ink, buff.ly, cutt.ly, goo.gl, is.gd, ow.ly, rb.gy, rebrand.ly, s.id, short.io, shorte.st, shorturl.at, t.co, t.ly, tiny.cc, tinyurl.com, v.gd
Risky TLDs (22)	buzz, cam, cf, cfd, click, ga, gq, icu, link, live, ml, monster, mov, quest, rest, sbs, support, tk, top, work, xyz, zip
Free hosting / tunnels (22)	000webhostapp.com, blogspot.com, duckdns.org, firebaseapp.com, framer.app, github.io, glitch.me, godaddysites.com, ipfs.io, netlify.app, ngrok-free.app, ngrok.io, pages.dev, r2.dev, square.site, trycloudflare.com, vercel.app, web.app, webflow.io, weebly.com, wixsite.com, workers.dev
Cloud user content (15)	appspot.com, azureedge.net, azurewebsites.net, backblazeb2.com, blob.core.windows.net, cloudflare-ipfs.com, digitaloceanspaces.com, dweb.link, firebasestorage.googleapis.com, forms.gle, googleusercontent.com, ipfs.io, sites.google.com, storage.googleapis.com, web.core.windows.net
Mail-provider click tracking (18)	awstrack.me, brevo.com, cmail19.com, cmail20.com, exacttarget.com, hs-sites.com, hubspotlinks.com, klclic.com, list-manage.com, mailchimp.com, mailgun.org, mandrillapp.com, mcusercontent.com, mlsend.com, rs6.net, sendgrid.net, sendibt3.com, sparkpostmail.com
Credential words (21)	account, auth, banking, billing, confirm, invoice, kyc, login, logon, otp, passwd, password, secure, sign-in, signin, suspended, unlock, update, verification, verify, wallet
Executable extensions (24)	apk, bat, cmd, com, cpl, dll, exe, hta, img, iso, jar, js, jse, lnk, msc, msi, ps1, reg, scr, vbe, vbs, vhd, vidx, wsf
Macro / HTML / archive extensions	docm, dotm, ppam, pptm, xlam, xls, xltm/htm, html, mht, mhtml, shtml, svg, xhtml/7z, ace, arj, bz2, cab, gz, rar, tar, xz, zip
Dangerous / ignored link schemes	data, file, javascript, vbscript/about, cid, mailto, sms, tel

Lookalike matching: a domain matches a brand if its look-alike skeleton (0→o, 1→l, rn→m, Cyrillic a→a, ...) equals the brand or contains it as a separate word; substring and one-typo matches are only used for brand names of at least 6 letters, because short names (apple, paytm) are parts of real words (applebees, paytv). Organisations can add up to 500 brands with EXTRA_BRANDS_FILE (validated JSON).

11.5 How the scorer combines everything

Each layer adds log-odds evidence; the sum is turned back into a probability:

```
logit = clip(text_logit, -4, +4)      # wording
      + 4 x header_risk              # header rules (max 1.0), email only
      + 4 x link_risk                # link/attachment rules (max 1.0)
      - trust_credit                 # verified official sender, email only
score = 1 / (1 + e^-logit)

email: text_logit = logit(p_email)
SMS:   text_logit = (logit(p_sms) + logit(p_email)) / 2
```

Parameter	Value	Why
Rule scale	4	A layer at its maximum risk adds +4 log-odds (50% becomes 98%). One DMARC failure (weight 0.35) adds +1.4.
Text clip	±4	Logistic regression is over-confident; without the clip, wording alone could veto hard technical evidence.

Parameter	Value	Why
SMS blend	average of logits	Chosen by the pre-registered experiment in section 16.
Verified-sender credit	-2	Only if DMARC passed at your mail provider for an official, non-free-mail brand domain with no spoofing signs.
Aligned-links credit	-1	Additionally, every link stays on that brand's domains (or its mailing provider).
Review threshold	0.5	Score at or above: label suspicious, action review.
Quarantine threshold	0.8 (default)	Score at or above: label phishing, action quarantine. Set with QUARANTINE_THRESHOLD in .env (0.5 to 1.0).

- Credits are capped (at most -3) so wording can still push a verified sender to **review**: scammers abuse real platforms too (fake invoices sent by paypal.com itself).
- Free-mail domains (gmail.com, outlook.com, Microsoft 365's default onmicrosoft.com tenants, ...) never earn trust: anyone can get an address there.
- **Fail-safe**: messages over the parser limits, unparseable input, internal errors and messages only partly analysed because of MIME limits always get **review**, never deliver.
- Every contribution is returned as a reason with its signed weight, sorted by size, so each verdict answers "why?". For SMS, each model's word weights count half. Reason texts are sanitised (no control, invisible or bidi characters).

12. Every threshold and fixed number

Every threshold, weight and limit in the system. All were fixed in advance; none was tuned on a test set. Rule weights are in 11.2 and 11.3.

Decision thresholds

Threshold	Value	Where	Effect
Review	score $\geq$ 0.5	scorer.REVIEW_THRESHOLD	label suspicious, action review
Quarantine	score $\geq$ 0.8 (default)	QUARANTINE_THRESHOLD in .env	label phishing, action quarantine. Allowed range 0.5 to 1.0; anything else stops start-up
Model metrics	probability $\geq$ 0.5	text_model._metrics	how precision, recall and FPR in the manifest are counted
"At quarantine" metrics	probability $\geq$ 0.8	manifest, scorer_metrics	the same metrics at the quarantine level
Too little text	fewer than 3 words	scorer.MIN_TEXT_TOKENS	the explanation says the wording could not be judged; the model's score for the little text there is still counts

Scorer and rule parameters

Parameter	Value	Where
Rule scale / text clip	4 / ± 4 log-odds	scorer.RULE_SCALE, TEXT_CLIP
Trust credits	-2 verified sender, -1 aligned links	scorer
Layer cap	risk = min(1, sum of weights)	header and link layers
Bulk recipients	$\geq$ 10 visible To + Cc	header_checks.BULK_RECIPIENTS
Fuzzy brand matching	brand names $\geq$ 6 letters, one edit	domains._MIN_FUZZY_LEN
Long URL / many subdomains	> 100 characters / $\geq$ 4 extra labels	url_checks
URLs checked per message	100	url_checks.MAX_URLS_CHECKED
SMS length / bare domains	5,000 characters / 20 domains	scorer.MAX_SMS_CHARS
Words in an explanation	top 5; reason text $\leq$ 300 characters	scorer.TOP_TERMS

Training and data parameters

Parameter	Value	Where
Train / test split	80% / 20%, stratified, seed 42	text_model
Honeypot time split	oldest 70% train / newest 30% test	HONEYPOT_TRAIN_FRACTION
Curated sample weight	email 20, SMS 3	CURATED_WEIGHT, SMS_CURATED_WEIGHT
English filter	≥ 5 function-word hits and $\geq 8\%$ of tokens	is_english()
Minimum text	email 20 characters, SMS 2 characters	loaders
De-duplication key	first 2,000 normalised characters	loaders
Text length	input cut at 40,000, normalised text capped at 20,000 characters	MAX_TEXT_CHARS
TF-IDF / LogReg	see 11.1 and section 13	build_pipeline()

Safety limits (parser, input, download)

Limit	Value	Why
Email size	5 MB default, max 25 MB (MAX_EMAIL_BYTES)	Oversized mail gets review, never deliver
Header section / one header	256 KB / 16,384 characters before parsing, 1,000 after cleaning	Header-flood denial of service
MIME parts / nesting / boundaries	100 / 10 / 2,000 boundary lines	MIME-bomb defence
Text / HTML budget	200,000 characters / 1,000,000 HTML characters	Slow-parse defence
Links / URL length / addresses	300 / 2,048 characters / 500	Bounded work per message
Repeated headers	50 (e.g. Received)	Bounded work
Download size	120 MB per file, 3 attempts	Corrupt or hostile downloads

Quality bars your trained models must meet (enforced by pytest)

Model	Test set	Bar
Email	In-distribution	recall ≥ 0.95 and false-positive rate ≤ 0.01
Email	Leave-one-corpus-out	false-positive rate at 0.8 ≤ 0.05 for every held-out corpus
Email	Modern test	recall ≥ 0.75 and false-positive rate ≤ 0.15
Email	Honeypot future test	recall ≥ 0.85
SMS	In-distribution	recall ≥ 0.90 and false-positive rate ≤ 0.01
SMS	Modern test	recall ≥ 0.75

SMS experiment rules

Rule	Value
Eligibility guard	public SMS test split: false-positive rate $\leq 1\%$ and recall ≥ 0.90 at 0.5
Selection	highest validation F1 at 0.5; ties: fewer validation false positives at 0.8; then the simpler variant (simplest first: V0, V1, V3, V2)
Synthetic template weight	1 (one row = one message)
Template hygiene (tests)	word overlap (Jaccard) with every evaluation message < 0.4 and no shared run of 5+ words

13. Model and training settings

Setting	Value	Why
Algorithm	Logistic Regression	Fast, robust, and every word has a visible weight (explainable)
C (inverse regularisation)	4.0	Moderate regularisation
class_weight	balanced	Phishing is the minority class (about 1 in 6 email rows, 1 in 13 SMS rows)
solver / max_iter	liblinear / 1000	Deterministic solver
random_state / seed	42	Reproducible splits and training
Curated sample weight	email 20, SMS 3	Fixed in advance (not tuned on the test set); about 3-4% influence each
Train / test split	80% / 20% stratified	For the CSV sources and the SMS corpus
Honeypot split	oldest 70% train / newest 30% test	By sample number = time order
Training rows (email)	6,125 phishing / 30,304 legitimate	Plus 56 curated train rows
Training rows (SMS)	347 smishing / 4,021 legitimate	Plus 56 curated train rows

Email source	Training rows
CEAS_08.csv	13,578
Enron.csv	11,517
Ling.csv	1,932
Nazario.csv	1,203
Nigerian_Fraud.csv	2,578
SpamAssasin.csv	3,277
phishing_pot	2,344
SMS source	Training rows
sms.tsv	155
sms_mendeley_5971.csv	4,213

14. How the models are evaluated

Test	What it measures	Data
In-distribution	Classic accuracy on unseen messages from the same sources	Stratified 20% of the CSV sources / SMS corpus
Honeypot future test	Catching next month's real phishing	Newest 30% of honeypot emails (time split)
Modern test	Today's lures and alarming-but-genuine notices	Held-out half of the 112 curated messages
Leave-one-corpus-out	False alarms on legitimate mail in an unfamiliar style	Each legitimate corpus held out in turn
Frozen SMS sets	False alarms on genuine transactional SMS (bank, UPI, OTP, delivery)	72 hand-written + 12 independent SMS, committed before any fix
Full system	The complete scorer as the app uses it	Honeypot .eml files with headers, legitimate test emails, modern test, SMS sets

Model results are stored in `models/manifest.json` every time you train; full-system results in `models/scorer_metrics.json`.

15. Expected results

A correct run reproduces these numbers **exactly** (if antivirus removed honeypot files, they may differ slightly).

15.1 Email model: in-distribution test

Metric	Value	Plain English
Emails tested	8,522	945 phishing, 7,577 legitimate
Accuracy	99.47%	8,477 of 8,522 correct
Precision	96.88%	When it says phishing, it is right this often
Recall	98.41%	Share of phishing caught
F1	0.9764	Balance of precision and recall
False-positive rate	0.40%	Share of legitimate emails wrongly flagged
ROC-AUC / PR-AUC	0.9997 / 0.9981	Ranking quality (1.0 = perfect)

	Predicted phishing	Predicted legitimate
Actually phishing (945)	930 caught	15 missed
Actually legitimate (7,577)	30 false alarms	7,547 correct

15.2 Honeypot future test (real 2026 phishing, never trained on)

Emails	Recall at 0.5	Recall at 0.8 (quarantine threshold)
1,005	87.9%	67.4%

15.3 Modern test (56 hand-written messages)

Accuracy	Precision	Recall	F1	False-positive rate	Errors by category
87.5%	88.9%	85.7%	0.873	10.7%	bec_giftcard 1, hinglish 2, in_upi 1, security_notice 1, utility 1, vendor_legit 1

15.4 Leave-one-corpus-out (false alarms on unfamiliar legitimate mail)

Held-out corpus	Emails	Flagged at 0.5	Flagged at 0.8	Phishing recall of probe
CEAS_08.csv	16,968	4.83%	2.17%	98.94%
Enron.csv	14,444	8.64%	1.13%	99.05%
Ling.csv	2,396	0.79%	0.17%	98.52%
SpamAssassin.csv	4,073	0.83%	0.10%	98.62%

15.5 Ablation: what each email dataset contributes

From python scripts/ablation.py (identical test sets for all three variants):

Training data	Future phishing recall	Modern recall / FPR	Old-source F1 / FPR
A: Nazario + Nigerian + Enron + SpamAssassin	40.60%	78.57% / 7.14%	0.9867 / 0.21%
B: A + CEAS_08 + Ling legitimate	29.95%	78.57% / 7.14%	0.9867 / 0.16%
C: B + honeypot (final model)	87.86%	85.71% / 10.71%	0.9862 / 0.30%

15.6 SMS model on its own (as stored in the manifest)

Test	Messages	Accuracy	Precision	Recall	F1	False-positive rate
SMS in-distribution	1,092	99.45%	96.55%	96.55%	0.9655	0.30%
Modern test	56	80.4%	79.3%	82.1%	0.807	21.4%

These are the SMS model alone. The app never uses it alone: SMS are scored with the SMS + email blend, whose results are in 15.7 and section 16. The SMS in-distribution test has only 87 smishing messages, so its recall moves in steps of about 1.1%.

15.7 The complete system (scorer)

From python scripts/evaluate_scorer.py. "Flagged" = review or quarantine (score at least 0.5); "quarantined" = score at least 0.8. Phishing rows show recall; legitimate rows show the false-positive rate.

Test set	Messages	Text alone: flagged	Scorer: flagged	Text alone: quarantined	Scorer: quarantined
Future honeypot phishing, English (real .eml)	1,005	87.9%	94.2%	67.4%	83.3%
Future honeypot phishing, all languages	5,465	73.2%	95.9%	37.6%	65.6%
Legitimate held-out emails (false alarms)	7,577	0.4%	0.5%	0.1%	0.1%
Modern test as email: phishing	28	85.7%	85.7%	64.3%	64.3%
Modern test as email: legitimate	28	10.7%	10.7%	3.6%	3.6%
Public SMS test split: smishing	87	90.8%	90.8%	54.0%	54.0%
Public SMS test split: legitimate	1,005	0.4%	0.4%	0.1%	0.1%
Modern test as SMS: phishing	28	85.7%	85.7%	78.6%	78.6%
Modern test as SMS: legitimate	28	21.4%	21.4%	3.6%	3.6%
Frozen SMS test: smishing	24	100.0%	100.0%	79.2%	79.2%
Frozen SMS test: genuine transactional	24	20.8%	20.8%	4.2%	4.2%
Independent real-format bank SMS	12	16.7%	16.7%	0.0%	0.0%

For SMS, "text alone" is the SMS + email blend without the link rules. Legitimate test emails come from CSVs without headers, so they are scored on text and links only: header rules could not be tested on legitimate mail because no public corpus of legitimate emails with full headers was available (section 21). The frozen SMS sets use reserved .test domains, which the link rules ignore, so for them scorer = text alone.

How to read this honestly: accuracy is flattering because about 89% of test emails are legitimate (a model that always says "legitimate" would already score about 89%). The most realistic figure is the **honeypot future recall**: real attacks newer than anything the model saw. A model trained only on the classic datasets (variant A) caught just 40.6% of them despite about 99% accuracy on its own test split.

16. The SMS false-positive experiment

The problem. Every legitimate SMS in the public corpora is a personal chat, so the SMS model learned "formal = scam": on its own it flags 37.5% of genuine transactional SMS (bank, UPI, OTP, delivery and bill alerts). No public corpus of legitimate transactional SMS exists, because they are personal data.

Protocol (fixed before any variant was trained)

- **Frozen evaluation sets**, committed before any fix: sms_transactional_eval.csv (36 genuine transactional SMS and 36 smishing in the same formats; per category 1 validation and 2 test messages) and sms_independent_bank.csv (12 real-format bank SMS by other developers). Masked phone numbers, reserved .test domains only.
- **Variants:** V0 = SMS model alone; V1 = SMS model retrained with the synthetic templates (training split only) and a normaliser that also drops masked numbers like XX1234; V2 = V1 averaged with the email model; V3 = V0 averaged with the email model (in log-odds).
- **Rules** (section 12): guard on the public SMS test split, then best validation F1. The test half and the independent set are scored only after the choice is fixed.
- **A leak was found and fixed.** The first run chose V1, but an overlap audit showed that some synthetic messages paraphrased evaluation messages (up to 8 shared consecutive words), which flattered V1 (test false-positive rate 8.3%). They were rewritten using overlap measures only, tests now enforce the overlap limits, and the unchanged experiment was run again. Run 2, the committed script, chose **V3**. The test half was looked at in both runs.

Results of run 2 (text score; the shipped variant is V3)

Measure	V0: SMS alone	V1: + synthetic	V2: V1 + email	V3: V0 + email (shipped)
Validation F1 at 0.5 (chooses)	0.667	0.741	0.833	0.750
Public split: FPR / recall at 0.5 (guard)	0.3% / 96.5%	0.3% / 95.3%	0.2% / 84.9%	0.4% / 90.8%
Eligible (guard passed)	yes	yes	no	yes
Frozen test: genuine SMS flagged (0.5 / 0.8)	37.5% / 4.2%	25.0% / 4.2%	20.8% / 0.0%	20.8% / 4.2%
Frozen test: smishing caught (0.5 / 0.8)	91.7% / 66.7%	95.8% / 87.5%	100.0% / 87.5%	100.0% / 79.2%
Independent bank SMS flagged (0.5 / 0.8)	2 / 2 of 12	3 / 0 of 12	0 / 0 of 12	2 / 0 of 12
Modern test: FPR / recall at 0.5	21.4% / 82.1%	7.1% / 89.3%	10.7% / 89.3%	21.4% / 85.7%
Public split: recall at 0.8	85.1%	73.3%	51.2%	54.0%

Honest reading. V3 beat V1 on validation by one message (F1 0.750 vs 0.741), and V1 did better on the modern hand-written test. The pre-registered choice was kept anyway: switching to whichever variant looks best on test data would make every reported number optimistic. V2 had the best validation score but failed the guard (it catches only 84.9% of public smishing). V3 cuts false alarms on genuine transactional SMS from 37.5% to 20.8% and catches all frozen-test smishing, but costs recall on the 2011-2022 public smishing at the 0.8 level (85.1% to 54.0%; 90.8% are still flagged for review). About 1 in 5 genuine transactional SMS is still flagged: see section 21.

Re-run everything with `python scripts/sms_experiment.py` (results in `models/sms_experiment.json`). The synthetic data is regenerated with `python scripts/generate_sms_templates.py`; tests check that it reproduces the committed file byte for byte and respects the overlap limits.

17. Using and inspecting your model

View all metrics

```
python -c "  
import json  
m = json.load(open('models/manifest.json'))['email_model']['metrics']  
print(json.dumps(m, indent=2))  
"
```

Replace `email_model` with `sms_model` for the SMS model.

Score your own text with the email model

```
python -c "  
from phishguard.text_model import load_model, TextModel  
tm = TextModel(load_model('email_model'))  
msg = 'Dear customer, your account will be suspended. Verify your password today.'  
print(round(tm.predict_proba(msg), 3), tm.top_terms(msg, 5))  
"
```

Expected output (verified): `1.0 [ ('your', 1.926), ('account', 1.34), ('dear', 1.113), ('verify', 0.841), ('dear customer', 0.777) ]`

Score a message with the complete system

```
python -m phishguard.scorer email tests/fixtures/phish_spoofed_sender.eml  
python -m phishguard.scorer sms "Your SBI account is blocked. Update KYC at sbi-kyc-update.in/verify"
```

Each prints a JSON verdict with score, label, action, components and the sorted reasons. Verified results: the fixture gets action: quarantine (score 1.0); the SMS gets action: quarantine (score 0.996; `sms_model_probability` 0.9027, `email_model_probability` 0.9935; reasons: `text_model`, `url_lookalike_domain`, `url_credential_words`).

See which words a model relies on

```
python -c "  
from phishguard.text_model import load_model  
import numpy as np  
m = load_model('email_model')  
words = m.named_steps['tfidf'].get_feature_names_out()  
coef = m.named_steps['clf'].coef_[0]  
print('phishing words:', [words[i] for i in np.argsort(coef)[-15:][::-1]])  
print('legit words: ', [words[i] for i in np.argsort(coef)[:15]])  
"
```

Always look at these lists after changing data (use `sms_model` for SMS). Names, dataset-specific words or language words at the top mean the model learned a shortcut. This is how every artefact in the stop-word list was found.

On Windows PowerShell, put each Python snippet into a file (e.g. `check.py`) and run `python check.py`.

18. Adding your own examples

Modern email and SMS examples

To teach both models lures you see (e.g. a new local scam), add rows to `data/curated/modern_lures.csv`:

```
id,split,label,category,text  
113,train,1,in_upi,"Your refund is pending. Enter your UPI PIN to receive Rs 2,000."  
114,test,1,in_upi,"Cashback of Rs 500 credited. Approve the collect request to claim."  
115,train,0,otp,"Your OTP is 123456. Never share it with anyone, including bank staff."
```

- `id` unique; `split` is `train` or `test`; `label` 1 = phishing, 0 = legitimate.
- Put examples of every category in **both** splits, and add legitimate look-alikes too, or the model learns that the topic itself means phishing.
- Never put real personal data, real links or real addresses in this file (it is committed to git; a test enforces no links or addresses).
- Then retrain both models (the file feeds both): `python scripts/bootstrap.py --force` and run `pytest`.

Your own transactional SMS (the real fix for SMS false alarms)

- Collect genuine bank, UPI, OTP and delivery SMS from **your own** phone only, with consent from anyone else involved. Mask account and phone numbers (XX1234) and remove names before saving.
- Keep them out of git (put them under `data/raw/`, which is gitignored), add them to the training side only, and keep a separate set you never train on to measure the result.
- Do not reuse `sms_transactional_eval.csv` or `sms_independent_bank.csv` for training or for tuning: they have already been used to choose a variant. Measure a new change on a **new** frozen set.
- To experiment with synthetic data, edit the templates in `scripts/generate_sms_templates.py`, regenerate, and re-run `python scripts/sms_experiment.py`. The tests will fail if a template copies wording from an evaluation message.

Rule of thumb: do not change settings until the test numbers look good; that silently turns your test set into training data. Decide what you will try and how you will choose before you look at results.

19. Troubleshooting

Symptom	Cause	Fix
[retry] ... truncated transfer or SHA-256 mismatch	Transfer cut short or corrupted	Nothing to do: the script retries up to 3 times and only accepts the pinned hash.
[fail] ... gave up after 3 attempts	Persistent corruption, or the mirror changed	Run the script again. If it persists, do not train: the data is not what this guide documents.
[fail] ... 403 or connection errors	Network or firewall blocks GitHub	Allow github.com and raw.githubusercontent.com, or use another network.
git is required for git-hosted datasets	git not installed or not on PATH	Install git (git-scm.com) and open a new terminal.
FileNotFoundError ... run scripts/download_data.py	Training started before the download finished	Run the download first; every line must be [done] or [ok].
ModelIntegrityError ... does not match its manifest hash	Model file changed or came from elsewhere	Run python scripts/bootstrap.py (retrains). Never bypass this check.
Your manifest SHA-256 differs from the author's	Expected: the saved vectorizer contains a memory address	Compare metrics, not hashes. git will show models/manifest.json as modified; that is fine.
Fewer than 3,349 phishing_pot rows	Antivirus removed honeypot files	Expected; metrics differ slightly. Don't add AV exclusions; a VM or WSL2 avoids it.
python: command not found or wrong version	Python missing or older than 3.11	Install Python 3.11+; on Linux and macOS use python3.
Activate.ps1 cannot be loaded	PowerShell execution policy	Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
Quality tests fail after your changes	Your data or settings lowered quality	Inspect top words (section 17); revert the change that caused it.
ModelIntegrityError: model file sms_model.joblib not found	The scorer loads both models (SMS uses both)	Train both: python scripts/bootstrap.py.
Out of disk space	About 4 GB needed	Free space; the honeypot alone is about 2.5 GB including git objects.

20. Metric glossary

Term	Meaning
TP / FP / TN / FN	Phishing caught / legitimate wrongly flagged / legitimate correctly passed / phishing missed
Accuracy	(TP + TN) / all. Misleading when classes are imbalanced.
Precision	TP / (TP + FP): how trustworthy a "phishing" verdict is
Recall	TP / (TP + FN): share of phishing caught
F1	Harmonic mean of precision and recall
False-positive rate (FPR)	FP / (FP + TN): share of legitimate messages flagged; what annoys users
ROC-AUC	Probability a random phishing message scores higher than a random legitimate one
PR-AUC	Average precision across thresholds; informative for the minority (phishing) class
Threshold	Score at or above which a message counts as phishing: 0.5 for metrics and review, 0.8 for quarantine
Log-odds (logit)	$\log(p / (1 - p))$. Evidence in log-odds can be added: +4 turns 50% into 98%
Shortcut learning	The model learns a dataset quirk (a name, a language, an era) instead of real phishing behaviour
Data leakage	Test information reaching training (duplicates in both splits, paraphrased test messages), inflating scores
Pre-registration	Fixing the candidates and the selection rule before seeing results, so the choice cannot be tuned to the test set
Validation / test set	Validation chooses between options; the test set only reports the final result

21. Known limitations

- **No modern legitimate email corpus exists publicly** (real inboxes are private). Legitimate training mail is from 2000-2008, so genuine modern "your account" notices can score high (3 of 28 legitimate modern-test messages flagged).

- **English only.** Non-English messages are not judged reliably by the text models; header and URL checks still apply to them.
- **Small hand-written test sets** (56 modern, 72 + 12 SMS messages): those figures are indicative, not precise. One message moves a rate by 3.6 to 8.3 percentage points.
- **SMS false alarms remain:** about 1 in 5 genuine transactional SMS is still flagged for review (20.8% of the frozen test, 21.4% of the modern legitimate test), because every legitimate training SMS is personal chat. Consented, anonymised transactional SMS from your own phone are the real fix.
- **The SMS blend trades recall at the quarantine level** on 2011-2022 public smishing (85.1% to 54.0% at 0.8; 90.8% still flagged at 0.5).
- Gift-card BEC, UPI and Hinglish scams are under-represented; some are still missed (15.3).
- **Header rules are untested on legitimate mail:** no public corpus of legitimate emails with full headers was available, so the full-system false-positive rate for email covers text and links only.
- The complete scorer quarantines 83.3% of future English phishing (94.2% flagged); the rest is either reviewed or missed.
- Hand-maintained lists (brands, shorteners, free hosts, risky TLDs) need periodic updates.

22. Licences and attribution

Data	Licence / terms
phishing_pot	CC BY-NC 4.0 (verified from the repository's LICENSE file): attribution, non-commercial use only
UCI SMS Spam Collection	CC BY 4.0 as listed by UCI (not re-checked by the author: UCI was unreachable)
SMS Phishing Dataset (Mishra and Soni)	CC BY 4.0 as listed on the Mendeley record (not re-checked: Mendeley was not opened); the GitHub mirror has no licence file
Kaggle compilation and its sources	Licence of the compilation not verified by the author (Kaggle was unreachable). Check the Kaggle page; each original corpus is a research corpus with its own terms.
GitHub mirror (rokibulroni)	Unofficial copy without a licence file; the original sources' terms apply
sms_independent_bank.csv	Example SMS from two MIT-licensed projects; both licences are reproduced in THIRD_PARTY_NOTICES.md
modern_lures.csv, sms_transactional_eval.csv, sms_templates.csv and all code	This repository's licence (MIT)
Datasets in section 7	Not used; licences not verified. Check each before use.

This repository never redistributes the downloaded datasets; each user downloads them and keeps them out of git. Cite the original authors when you publish results.